



in collaboration with



Title:

Lab 3 Task

Course:

ST5003CMD-Operating Systems, Security and Network

Institution:

Softwarica College of IT and E-Commerce

Submitted By:

Sandip Rawal (240684)

Submitted To

Mr.Rupak Rajbanshi

Submission Date:

September 26 , 2026

Table of Contents:

I. Introduction.....	4
II, Task 1: The Long Running Process.....	5
A. C Source Code.....	5
B. Terminal Execution and Verification.....	6
III. Task 2: Process Identity (PID and PPID).....	7
A. C Source Code.....	7
B. Terminal Execution and Verification.....	7
IV. Task 3: Exit Codes and OS Feedback.....	8
A. C Source Code.....	8
B. Terminal Execution and Verification.....	8
V. Task 4: Standard I/O Streams.....	9
A. C Source Code.....	9
B. Terminal Execution.....	9
VI. Task 5: Conditional Execution and Termination.....	10
A. C Source Code.....	10
B. Terminal Execution and Verification.....	10
VII. Flow Diagrams.....	11
VIII. Final Verification.....	12
IX. Conclusion.....	13

List Of Figures:

Figure 1: C Code in nano task1_alive.....	5
Figure 2: Running Task 1 in the Background.....	6
Figure 3: Monitoring the Running Task 1 Process.....	6
Figure 4: C Code in nano task2_identity.....	7
Figure 5: Displaying and Verifying PID and PPID.....	7
Figure 6: C Code in nano task3_exit.....	8
Figure 7: Successful and Failed Execution and Exit Code.....	8
Figure 8: C Code in nano task4_input.....	9
Figure 9: Standard input and output.....	9
Figure 10: C Code in nano task5_control.....	10
Figure 11: Continue option and Successful Exit with Exit Option and Failure Status.	10
Figure 12: Flow Diagrams Of Five Tasks.....	11
Figure 13: Completed Lab 3 Source Files.....	12

I. Introduction

An operating system manages processes, memory, files, and other computer resources while programs are running. A process is a running instance of a program, and the Linux operating system assigns each process a unique Process ID (PID). This lab provides practical experience with how C programs interact with the Linux operating system and how the operating system manages their execution. The lab uses C programs and Linux terminal commands to investigate process lifecycles, process identity, exit codes, standard input and output, and conditional process termination.

System calls and functions such as `getpid()`, `getppid()`, and `sleep()` are used to obtain process information and control execution. The `return` statement is also used to send a status code back to the operating system. Through these activities, the difference between a C program and an active operating-system process can be observed directly. Terminal commands such as `ps` and `echo $?` are used to verify the behaviour of the programs.

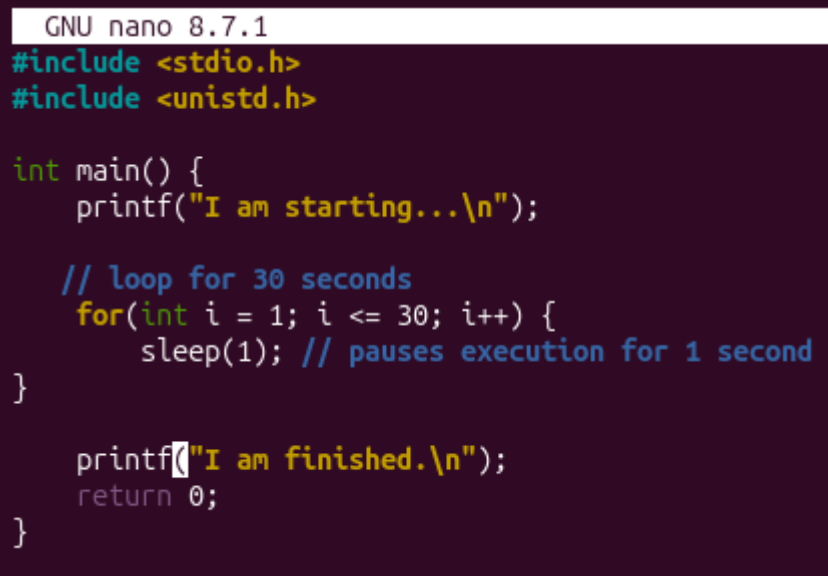
II, Task 1: The Long Running Process

This task demonstrates that a running program becomes an active process that can be monitored by Linux. The program runs for **30 seconds**, allowing us to inspect it from another terminal. This follows the lab's explanation of long-running processes.

A. C Source Code

We create a program that runs for 30 seconds using a loop and sleep().

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ nano task1_alive.c
```



```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting...\n");

    // loop for 30 seconds
    for(int i = 1; i <= 30; i++) {
        sleep(1); // pauses execution for 1 second
    }

    printf("I am finished.\n");
    return 0;
}
```

Figure 1: C Code in nano task1_alive

B. Terminal Execution and Verification

To make this interesting, we run the program in the background using `&` so we can use a second terminal to inspect it while it's alive.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ gcc task1_alive.c -o task1
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ ls
task1  task1_alive.c
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ ./task1 &
[1] 5045
I am starting...
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ I am finished.
```

Figure 2: Running Task 1 in the Background

Monitoring in Second Terminal

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ ps aux | grep task1
sandip-+  5045  0.0  0.0  2768  1684 pts/0    S   16:49   0:00 ./task1
```

Figure 3: Monitoring the Running Task 1 Process

III. Task 2: Process Identity (PID and PPID)

Every Linux process has a **PID**, while the process that created it has a **PPID**. This task uses `getpid()` and `getppid()` to obtain those values directly from the operating system.

A. C Source Code

The program uses `getpid()` and `getppid()` to display its own Process ID (PID) and Parent Process ID (PPID).

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ nano task2_identity.c
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();

    // Get Parent Process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}
```

Figure 4: C Code in nano task2_identity

B. Terminal Execution and Verification

The displayed PID and PPID are verified using the Linux `ps` command.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ gcc task2_identity.c -o task2
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ ./task2 &
[1] 8218
My PID is: 8218
My Parent PID is: 4995
Sleeping for 20 seconds...
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ ps -p 8218 -o pid,ppid,cmd
  PID   PPID  CMD
[1]+  Done                  ./task2
```

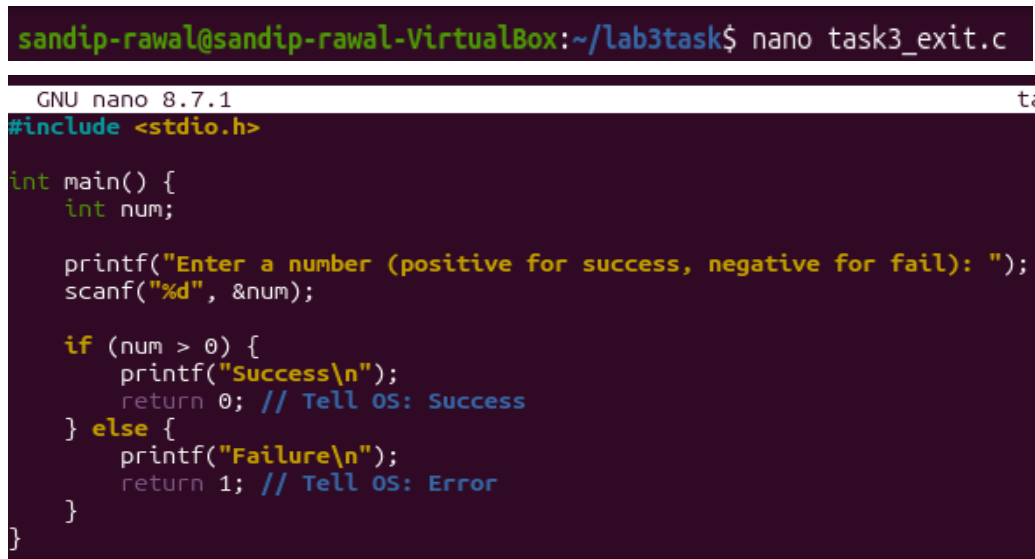
Figure 5: Displaying and Verifying PID and PPID

IV. Task 3: Exit Codes and OS Feedback

A program can send an exit status to the operating system when it finishes. In this task, `0` represents success and `1` represents failure, while the shell's `$?` variable displays the previous command's exit status.

A. C Source Code

The program checks whether the entered number is positive or negative and returns `0` for success or `1` for failure.



```
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ nano task3_exit.c

GNU nano 8.7.1                                     ta
#include <stdio.h>

int main() {
    int num;

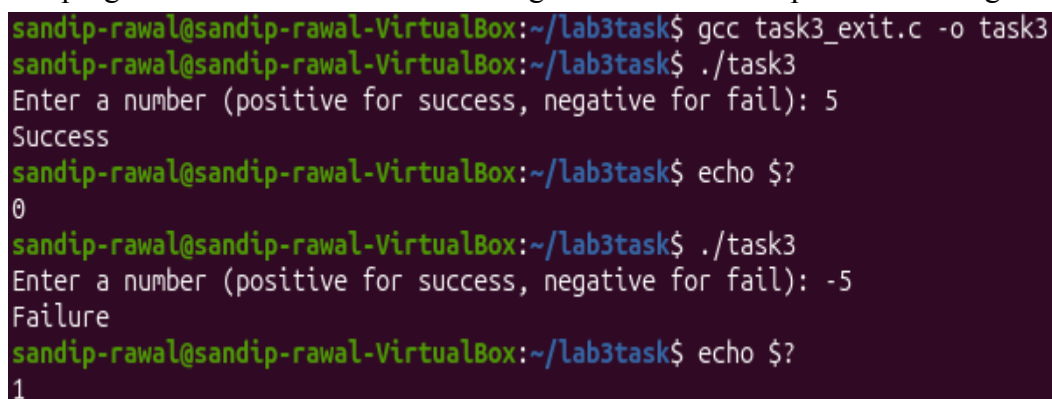
    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success\n");
        return 0; // Tell OS: Success
    } else {
        printf("Failure\n");
        return 1; // Tell OS: Error
    }
}
```

Figure 6: C Code in nano task3_exit

B. Terminal Execution and Verification

The program's exit status is checked using `echo $?` after both positive and negative inputs.



```
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ gcc task3_exit.c -o task3
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ ./task3
Enter a number (positive for success, negative for fail): 5
Success
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ echo $?
0
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ ./task3
Enter a number (positive for success, negative for fail): -5
Failure
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ echo $?
1
```

Figure 7: Successful and Failed Execution and Exit Code

V. Task 4: Standard I/O Streams

Linux provides standard input and output streams. In this program, `scanf()` receives input from `stdin`, while `printf()` sends output to `stdout`.

A. C Source Code

The program uses `scanf()` to receive the user's name through standard input and `printf()` to display the output.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ nano task4_input.c

GNU nano 8.7.1
#include <stdio.h>

int main () {
    char name[50];

    printf("Enter your name: ");

    // Read String from standard input
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS Class.\n", name);

    return 0;
}
```

Figure 8: C Code in nano task4_input

B. Terminal Execution

The program is compiled and executed to demonstrate interaction through standard input and output.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ gcc task4_input.c -o task4
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ ./task4
Enter your name: Sandip
Hello, Sandip! Welcome to OS Class.
```

Figure 9: Standard input and output

VI. Task 5: Conditional Execution and Termination

This task demonstrates how a process can make a decision based on user input and then terminate with different exit codes. It also displays its current PID using `getpid()`.

A. C Source Code

The program displays its PID and uses the user's choice to either continue and return 0 or exit and return 1.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ nano task5_control.c
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    // Print PID at Start
    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; // Success
    } else {
        printf("Exiting...\n");
        return 1; // Failure/Abort
    }
}
```

Figure 10: C Code in nano task5_control

B. Terminal Execution and Verification

The program is tested with both 1 and 0, and the resulting exit codes are verified using `echo $?`.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ gcc task5_control.c -o task5
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ ./task5
Current PID: 11999
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ echo $?
0
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ ./task5
Current PID: 12003
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ echo $?
1
```

Figure 11: Continue option and Successful Exit with Exit Option and Failure Status

VII. Flow Diagrams

Below is the flow Diagram of everything that was carried out during lab 3 tasks.

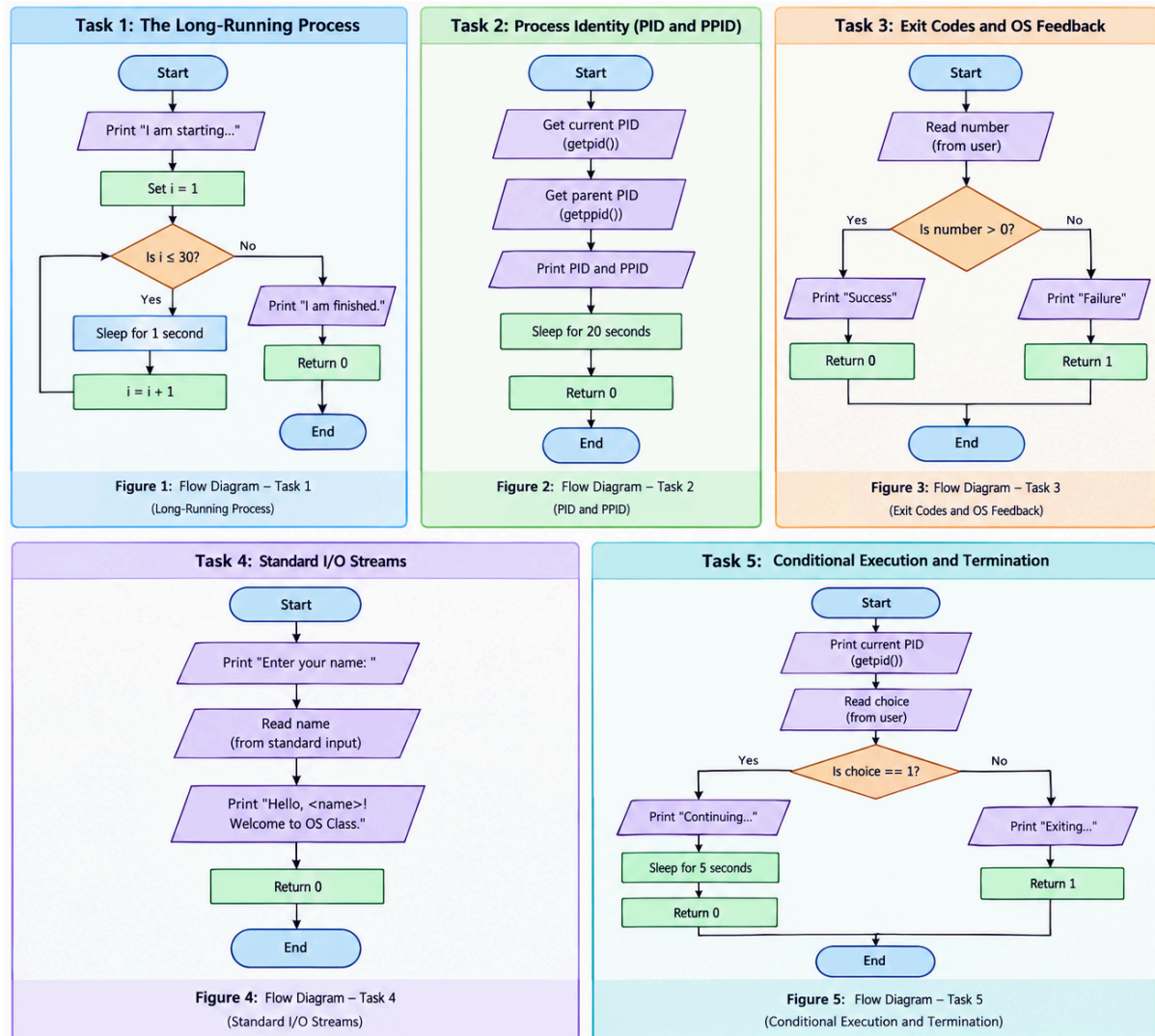


Figure 12: Flow Diagrams Of Five Tasks

VIII. Final Verification

Completed all tasks of Lab 3 below is the proof needed for final verification.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab3task$ ls -1
task1
task1_alive.c
task2
task2_identity.c
task3
task3_exit.c
task3_exit.c.save
task4
task4_input.c
task5
task5_control.c
```

Figure 13: Completed Lab 3 Source Files

IX. Conclusion

This lab provided practical understanding of how C programs operate as processes under the Linux operating system. The five tasks demonstrated different aspects of process management and OS interaction. The first task showed how a long-running process can execute in the background and be monitored using the Linux `ps` command.

The second task demonstrated process identity by retrieving and verifying the Process ID (PID) and Parent Process ID (PPID). The third task showed how a program communicates its completion status to the operating system using exit codes, where `0` represents success and `1` represents failure. The fourth task demonstrated standard input and output through `scanf()` and `printf()`.

Finally, the fifth task combined process identification, user input, conditional execution, delay, and termination status. Overall, the lab connected C programming concepts with practical Linux process management and showed how the operating system monitors and interacts with running programs.